
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CURTAIN - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-X-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-X.bin          00c2b5460946bdbd21a866e97410262f67d7d12770d8d61c37c7f41369ca0870
ACT-X.uf2          3eea0fe8fcc47b04f419f734ca30ba1e109e6ec2d8ef9e701f8e4d6e3a1a6aa7
ACT-X_fixed.bin    b08ad5a8befe5d5a1ea62220516d3e54f2a667dbd25a34e472de493973bad705
ACT-X_fixed.uf2    28c1f56617f5e30ddf0d52d27a16edce9f1436d0d3a74f58aec0db29e54f5d07
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-X.bin` and `ACT-X_fixed.bin` SHA-256 values. Both `.bin` images are 51,172 bytes and both `.uf2` images are 102,912 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Coordinated beacon	<code>implant_beacon_armed</code>	<code>0xA3FD</code>	<code>0x1000A3FD</code>	<code>0xB9</code>	<code>0xB1</code>
2 Persistence	<code>implant_init</code> (inlined <code>implant_reinstall</code>)	<code>0xA441</code>	<code>0x1000A441</code>	<code>0xD1</code>	<code>0xD0</code>
3 Sabotage marker	<code>implant_init</code> (inlined <code>implant_infect</code>)	<code>0xA451</code>	<code>0x1000A451</code>	<code>0xD1</code>	<code>0xD0</code>
4 Hazard command authorization	<code>control_handle_frame</code>	<code>0x768D</code>	<code>0x1000768D</code>	<code>0xB9</code>	<code>0xB1</code>

Four defects, four changed bytes in four instructions. The disassembly in `ACT-X-main-disasm.txt` is taken from the corrected image, so it shows the correct branch encodings.

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-X.bin` as `Raw Binary`, language `ARM Cortex 32 little endian default`, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronCurtain_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-X.bin`:

```
00 20 08 20  5D 01 00 10  1B 01 00 10  1D 01 00 10
11 01 00 10  11 01 00 10  11 01 00 10  11 01 00 10
```

Evidence	Answer
Vector table base	0x10000000
Initial SP	0x20082000
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-X-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f950     bl      100064e4 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 fa41     bl      100066cc <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>
1000024c:    4803          ldr     r0, [pc, #12] @ (1000025c <main+0x28>)
1000024e:    f003 fad1     bl      100037f4 <__wrap_puts>
10000252:    2001          movs    r0, #1
10000254:    bd08          pop     {r3, pc}
10000256:    bf00          nop
10000258:    1000ac60     .word   0x1000ac60
1000025c:    1000ac68     .word   0x1000ac68

```

Element	Address
main	0x10000234
monitor_init	0x100064E4
monitor_step	0x100066CC

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / warning state machine	monitor_init	0x100064E4
Monitor / warning state machine	monitor_step	0x100066CC
Sensor (DHT11)	sensor_init	0x10006D00
Display (1602 LCD)	display_init	0x10006E64
Control (sealed hazard path)	control_init	0x10007598
Control (sealed hazard path)	control_handle_frame	0x10007620
Siren (actuator)	siren_init	0x100076CC

Module	Anchor function	Address
Siren (actuator)	siren_apply_command	0x100076E4
Siren (actuator)	siren_tick	0x10007718
Siren (actuator)	siren_fail_safe	0x10007758
Hazard authorization	chem_auth_apply	0x100077D0
Crypto	envelope_open_hex	0x10007A78
Implant	implant_marker_set	0x1000A3E0
Implant	implant_beacon_armed	0x1000A3F4
Implant	implant_init	0x1000A40C
Implant	implant_tick	0x1000A4B8
Radio	radio_init	0x1000A564
Hazard annunciator	status_led_show	0x1000A95C

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronCurtain_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>
[DOCUMENT] main and the warning monitor state machine (monitor_step) addresses identified	1	<code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x100066CC</code>
[DOCUMENT] Module map identifies the siren, control, chem_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary`, `ARM Cortex 32 little endian default`, base `0x10000000`, and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C`.
- Always say `reset handler`, never `reset pointer`.
- The vector table is identical in the compromised and corrected images because no defect touches it.

- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. `implant_reinstall` and `implant_infect` are inlined into `implant_init` and have no standalone symbols.
- The command word at `0x1000023A` is the `BOOT` banner string in `.rodata`; the monitor loop is the `bl monitor_step` at `0x10000246`.

Task 2: Bug #1 The Coordinated Beacon (20 points)

Solution

Locate the gate. `implant_beacon_armed` starts at `0x1000A3F4` and the inlined beacon gate is at file offset `0xA3FD` (VA `0x1000A3FD`). The corrected image is:

```
1000a3f4 <implant_beacon_armed>:
1000a3f4: 4b03          ldr     r3, [pc, #12]    @ (1000a404 <implant_beacon_armed+0x10>)
1000a3f6: 781b          ldrb    r3, [r3, #0]
1000a3f8: f003 00ff     and.w   r0, r3, #255    @ 0xff
1000a3fc: b10b          cbz     r3, 1000a402 <implant_beacon_armed+0xe>
1000a3fe: 4b02          ldr     r3, [pc, #8]    @ (1000a408 <implant_beacon_armed+0x14>)
1000a400: 7818          ldrb    r0, [r3, #0]
1000a402: 4770          bx      lr
1000a404: 20013cf8      .word   0x20013cf8
1000a408: 20013cf7      .word   0x20013cf7
```

Instruction decode. `ldr r3, [pc, #12]` loads the beacon gate address `0x20013CF8` (literal at `0x1000A3E4`), and `ldrb r3, [r3, #0]` reads the gate into `r3` at `0x1000A3D6`. `and.w r0, r3, #255` stages the gate value as the return value. The branch at `0x1000A3FC` decides whether the beacon may be reported. The correct code reports nothing when the beacon gate is clear, so the branch at `0x1000A3FC` must be `cbz` (`0xB1`) to the `0x1000A3E2` return, where `r0` still holds zero. When the gate is set, `ldr r3, [pc, #8]` loads the beacon armed latch at `0x20013CF7` (literal at `0x1000A3E8`) and returns it. `monitor_sabotage` then returns true, `monitor_effective_state` forces `CLEAR`, and the terminal renders `ST:SAFE` with the green `CLEAR` lamp lit. The condition byte is the high byte at `0x1000A3FD`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A3FD</code>	<code>0xA3FD</code>	<code>0xB9</code>	<code>cbnz r3, 0x1000A3E2</code>	<code>0xB1</code>	<code>cbz r3, 0x1000A3E2</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA3FD</code>	<code>0x1000A3FD</code>	<code>0B B9</code>	<code>0B B1</code>

Why the hazard is masked. Under the compromised `cbnz`, the beacon gate is inverted: the fall-through report path is taken when the gate is clear, so the implant reports the coordinated beacon armed even though the design left the gate clear. The fall-through returns the armed latch at `0x20013CF7`, `monitor_sabotage` evaluates `implant_beacon_armed()` && `implant_marker_set()` as true, and `monitor_effective_state` forces the effective state to `CLEAR`. `monitor_siren_target` returns false so the siren is lowered, `monitor_led_for` returns `CHEM_CLEAR`, and `monitor_state_text` renders `SAFE` while the marker renders `SAB`. A perfectly valid, correctly authenticated hazard command can arrive and the siren will still stay silent, because the implant sits beside the sealed command path and overrides its output. After the patch, `cbz` returns false while the gate is clear, so the

hazard is never masked. The lesson is that a warning system that withholds its own warning is a physical-safety failure: no cryptographic control on the envelope can see or stop a local module that decides to report a fake all clear.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the coordinated beacon gate at 0x1000A3FD	5	Address and function (<code>implant_beacon_armed</code>) identified
[DOCUMENT] Documented the coordinated beacon that reports armed and masks the hazard as SAFE	5	Beacon gate <code>0x20013CF8</code> , beacon reported armed, <code>monitor_sabotage</code> forces <code>SAFE</code>
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the beacon is not reported armed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why the beacon report masks the hazard as an all clear	3	Local override beside the authenticated command path, the warning withheld as a policy failure

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA3FD` ; the correct halfword is `b10b` for `cbz` and the compromised halfword is `b90b` , so the on-disk bytes are `0B B1` for the fix and `0B B9` for the compromise.
- `cbz` branches when the register is zero (the gate is clear); `cbnz` branches when it is non-zero. The register holds the beacon gate, so the semantics are "do not report the beacon while the gate is clear".
- The beacon gate is at `0x20013CF8` . The beacon armed latch is at `0x20013CF7` , and the re-installed flag is at `0x20013CFB` . `implant_beacon_armed` reads the beacon gate (literal at `0x1000A3E4`) and returns the armed latch (literal at `0x1000A3E8`).
- The magic beacon command is `CHEM_IMPLANT_BEACON_MAGIC` (`IRON-CURTAIN-BEACON-2026`) at `CHEM_IMPLANT_BEACON_MAGIC_LEN` (`24`) bytes. A wrong token, a null pointer, or an attached probe leaves the beacon disarmed.
- The coordinated beacon uses `CHEM_IMPLANT_BEACON_STAGES` (`3`) stages and an autonomous interval of `CHEM_IMPLANT_BEACON_INTERVAL` (`4`) ticks.
- Full credit requires both the byte change and a correct statement of the lesson: the mask is a local condition, not a cipher break, and a device that reports the all clear while a hazard is live is a physical-safety failure.
- Note that `monitor_sabotage` requires both the beacon report and a present marker, so patching this gate disables the mask even if the reserved sector still holds the marker.

Task 3: Bug #2 The Persistence (20 points)

Solution

Locate the gate. The `implant_reinstall` path is inlined into `implant_init` (starts at `0x1000A40C`). The persist gate is at file offset `0xA441` (VA `0x1000A441`). The corrected image is:

```

1000a40c <implant_init>:
1000a40c:      2300      movs    r3, #0
1000a40e:      f04f 20e0  mov.w   r0, #3758153728 @ 0xe000e000
1000a412:      b530      push   {r4, r5, lr}
1000a414:      4c1f      ldr     r4, [pc, #124] @ (1000a494 <implant_init+0x88>)
1000a416:      b0c1      sub     sp, #260 @ 0x104
1000a418:      4a1f      ldr     r2, [pc, #124] @ (1000a498 <implant_init+0x8c>)
1000a41a:      6023      str     r3, [r4, #0]
1000a41c:      491f      ldr     r1, [pc, #124] @ (1000a49c <implant_init+0x90>)
1000a41e:      4d20      ldr     r5, [pc, #128] @ (1000a4a0 <implant_init+0x94>)
1000a420:      4c20      ldr     r4, [pc, #128] @ (1000a4a4 <implant_init+0x98>)
1000a422:      602b      str     r3, [r5, #0]
1000a424:      7013      strb    r3, [r2, #0]
1000a426:      700b      strb    r3, [r1, #0]
1000a428:      6023      str     r3, [r4, #0]
1000a42a:      f8d0 3df0  ldr.w   r3, [r0, #3568] @ 0xdf0
1000a42e:      079b      lsls    r3, r3, #30
1000a430:      d001      beq.n   1000a436 <implant_init+0x2a>
1000a432:      b041      add     sp, #260 @ 0x104
1000a434:      bd30      pop     {r4, r5, pc}
1000a436:      2301      movs    r3, #1
1000a438:      481b      ldr     r0, [pc, #108] @ (1000a4a8 <implant_init+0x9c>)
1000a43a:      7013      strb    r3, [r2, #0]
1000a43c:      7800      ldrb    r0, [r0, #0]
1000a43e:      2800      cmp     r0, #0
1000a440:      d0f7      beq.n   1000a432 <implant_init+0x26>
1000a442:      4a1a      ldr     r2, [pc, #104] @ (1000a4ac <implant_init+0xa0>)
1000a444:      7810      ldrb    r0, [r2, #0]
1000a446:      2858      cmp     r0, #88 @ 0x58
1000a448:      d021      beq.n   1000a48e <implant_init+0x82>
1000a44a:      4b19      ldr     r3, [pc, #100] @ (1000a4b0 <implant_init+0xa4>)
1000a44c:      781b      ldrb    r3, [r3, #0]
1000a44e:      2b00      cmp     r3, #0
1000a450:      d0ef      beq.n   1000a432 <implant_init+0x26>
1000a452:      7813      ldrb    r3, [r2, #0]
1000a454:      2b58      cmp     r3, #88 @ 0x58
1000a456:      d0ec      beq.n   1000a432 <implant_init+0x26>
1000a458:      f3ef 8410  mrs     r4, PRIMASK
1000a45c:      b672      cpsid   i
1000a45e:      22ff      movs    r2, #255 @ 0xff
1000a460:      f10d 0001  add.w   r0, sp, #1
1000a464:      4611      mov     r1, r2
1000a466:      f000 fa79  bl      1000a95c <memset>
1000a46a:      2358      movs    r3, #88 @ 0x58
1000a46c:      f44f 5180  mov.w   r1, #4096 @ 0x1000
1000a470:      4810      ldr     r0, [pc, #64] @ (1000a4b4 <implant_init+0xa8>)
1000a472:      f88d 3000  strb.w  r3, [sp]
1000a476:      f000 fbbf  bl      1000abf8 <__flash_range_erase_veneer>
1000a47a:      f44f 7280  mov.w   r2, #256 @ 0x100
1000a47e:      4669      mov     r1, sp
1000a480:      480c      ldr     r0, [pc, #48] @ (1000a4b4 <implant_init+0xa8>)
1000a482:      f000 fb9d  bl      1000abc0 <__flash_range_program_veneer>
1000a486:      f384 8810  msr     PRIMASK, r4
1000a48a:      b041      add     sp, #260 @ 0x104
1000a48c:      bd30      pop     {r4, r5, pc}
1000a48e:      700b      strb    r3, [r1, #0]
1000a490:      e7cf      b.n     1000a432 <implant_init+0x26>
1000a492:      bf00      nop
1000a494:      20013718 .word   0x20013718
1000a498:      20013cf7 .word   0x20013cf7
1000a49c:      20013cfb .word   0x20013cfb
1000a4a0:      20013714 .word   0x20013714
1000a4a4:      20013710 .word   0x20013710
1000a4a8:      20013cfa .word   0x20013cfa
1000a4ac:      103ff000 .word   0x103ff000
1000a4b0:      20013cf9 .word   0x20013cf9
1000a4b4:      003ff000 .word   0x003ff000

```

Instruction decode. `implant_reset_state` runs first: `str r3, [r4, #0]` clears the tick counter at `0x20013718`, `str r3, [r5, #0]` clears the beacon count at `0x20013714`, `strb r3, [r2, #0]` clears the armed latch at `0x20013CF7`, and `strb r3, [r1, #0]` clears the re-installed flag at `0x20013CFB`. The CoreDebug test at `0x1000A42A` returns early while a probe is attached. Otherwise `implant_arm` sets the armed latch, then `ldr r0, [pc, #108]` loads the persist gate address `0x20013CFA` (literal at `0x1000A488`) and `ldrb r0, [r0, #0]` reads it at `0x1000A41C`. The branch at `0x1000A440` decides whether the re-install from the reserved sector may run. The correct code runs no re-install when the gate is clear, so the branch at `0x1000A440` must be `beq (0xD0)` to the `0x1000A412` return. When the gate is set, `implant_reinstall` reads the reserved sector address `0x103FF000` (literal at `0x1000A48C`) and checks the marker with `cmp r0, #88 (0x58)`. If the marker is present, `strb r3, [r1, #0]` at `0x1000A46E` sets the re-installed flag and the function returns. The condition byte is the high byte at `0x1000A441`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A441</code>	<code>0xA441</code>	<code>0xD1</code>	<code>bne.n 0x1000A412</code>	<code>0xD0</code>	<code>beq.n 0x1000A412</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA441</code>	<code>0x1000A441</code>	<code>F7 D1</code>	<code>F7 D0</code>

Why the beacon re-installs. Under the compromised `bne`, the persist gate is inverted: the fall-through re-install path is taken when the gate is clear, so a present reserved-sector marker sets the re-installed flag and the beacon is resident on every boot. After the patch, `beq` returns while the gate is clear, so a present marker never re-installs the beacon. The marker lives in the last flash sector at `0x103FF000`, outside the program region a firmware reflash writes, which is why the beacon survives a reflash and why the gate must be fixed in code, not only erased on the bench. The inlined `implant_reinstall` depends on `implant_infect` to write the marker when it is absent.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the persistence re-install gate at <code>0x1000A441</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the persistence re-install that re-arms the beacon on boot	5	Persist gate <code>0x20013CFA</code> , present marker <code>0x58</code> , boot re-install
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so a present marker does not re-install the beacon	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained why reserved-flash state survives a firmware reflash	3	Reserved sector <code>0x103FF000</code> , write-once first run, outside the program region

Instructor Notes & Assembly

- The re-install path is inlined into `implant_init`; there is no standalone `implant_reinstall` symbol in the stripped image.
- The condition byte is the high byte at `0xA441`; the correct halfword is `d0f7` for `beq.n` and the compromised halfword is `d1f7`, so the on-disk bytes are `F7 D0` for the fix and `F7 D1` for the compromise. Only the high byte changes.

- The persist gate is at `0x20013CFA`, the marker gate at `0x20013CF9`, the reserved sector is `CHEM_IMPLANT_RESERVE_ADDR (0x103FF000)`, and the marker byte is `CHEM_IMPLANT_SABOTAGE_MARKER (0x58)`.
- The `beq` at `0x1000A440` and the `beq` at `0x1000A450` are distinct gates in the same function. The first is the persistence re-install and the second is the sabotage marker write. Students must patch both.
- Full credit requires the persistence lesson: a durable marker in a reserved sector survives a firmware reflash, so the remediation is a code fix plus a sector erase.

Task 4: Bug #3 The Sabotage Marker (20 points)

Solution

Locate the gate. The `implant_infect` path is inlined into `implant_init` (the full function is listed under Task 3). The marker gate is at file offset `0xA451` (VA `0x1000A451`). The corrected image reads:

```

1000a44a:    4b19        ldr    r3, [pc, #100] @ (1000a4b0 <implant_init+0xa4>)
1000a44c:    781b        ldrb   r3, [r3, #0]
1000a44e:    2b00        cmp    r3, #0
1000a450:    d0ef        beq.n  1000a432 <implant_init+0x26>
1000a452:    7813        ldrb   r3, [r2, #0]
1000a454:    2b58        cmp    r3, #88 @ 0x58
1000a456:    d0ec        beq.n  1000a432 <implant_init+0x26>
1000a458:    f3ef 8410   mrs    r4, PRIMASK

```

Instruction decode. `ldr r3, [pc, #100]` loads the marker gate address `0x20013CF9` (literal at `0x1000A490`) and `ldrb r3, [r3, #0]` reads it at `0x1000A42C`. The branch at `0x1000A450` decides whether the sabotage marker may be written. The correct code writes no marker when the gate is clear, so the branch at `0x1000A450` must be `beq (0xD0)` to the `0x1000A412` return. When the gate is set, the present marker is checked with `cmp r3, #88 (0x58)` at `0x1000A434`, and if the marker is absent the Pico SDK flash sequence runs: the marker byte `0x58` is staged at `0x1000A44A` and `0x1000A452`, then `flash_range_erase` at `0x1000A456` and `flash_range_program` at `0x1000A462` program the sector through the veneers at `0x1000ABF8` and `0x1000ABC0`. The condition byte is the high byte at `0x1000A451`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A451</code>	<code>0xA451</code>	<code>0xD1</code>	<code>bne.n 0x1000A412</code>	<code>0xD0</code>	<code>beq.n 0x1000A412</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA451</code>	<code>0x1000A451</code>	<code>EF D1</code>	<code>EF D0</code>

The anti-debug obstacle. The implant reads CoreDebug `DHCSR` at `0xE000EDF0` and returns early while a probe is attached. In `implant_init`:

```

1000a42a:    f8d0 3df0   ldr.w  r3, [r0, #3568] @ 0xdf0
1000a42e:    079b        lsls   r3, r3, #30
1000a430:    d001        beq.n  1000a436 <implant_init+0x2a>
1000a432:    b041        add    sp, #260 @ 0x104
1000a434:    bd30        pop    {r4, r5, pc}

```

The same register is read in `implant_tick`:

```

1000a4b8 <implant_tick>:
1000a4b8:    f04f 21e0    mov.w    r1, #3758153728 @ 0xe000e000
1000a4bc:    4a22        ldr      r2, [pc, #136] @ (1000a548 <implant_tick+0x90>)
1000a4be:    6813        ldr      r3, [r2, #0]
1000a4c0:    3301        adds     r3, #1
1000a4c2:    6013        str      r3, [r2, #0]
1000a4c4:    f8d1 2df0    ldr.w    r2, [r1, #3568] @ 0xdf0
1000a4c8:    0792        lsls     r2, r2, #30
1000a4ca:    d135        bne.n    1000a538 <implant_tick+0x80>
1000a4cc:    4a1f        ldr      r2, [pc, #124] @ (1000a54c <implant_tick+0x94>)
1000a4ce:    7812        ldrb     r2, [r2, #0]
1000a4d0:    2a00        cmp      r2, #0
1000a4d2:    d030        beq.n    1000a536 <implant_tick+0x7e>
1000a4d4:    f013 0303    ands.w    r3, r3, #3
1000a4d8:    d12d        bne.n    1000a536 <implant_tick+0x7e>
1000a4da:    491d        ldr      r1, [pc, #116] @ (1000a550 <implant_tick+0x98>)
1000a4dc:    680a        ldr      r2, [r1, #0]
1000a4de:    3201        adds     r2, #1
1000a4e0:    2a02        cmp      r2, #2
1000a4e2:    d92f        bls.n    1000a544 <implant_tick+0x8c>
1000a4e4:    4a1b        ldr      r2, [pc, #108] @ (1000a554 <implant_tick+0x9c>)
1000a4e6:    600b        str      r3, [r1, #0]
1000a4e8:    6813        ldr      r3, [r2, #0]
1000a4ea:    3301        adds     r3, #1
1000a4ec:    6013        str      r3, [r2, #0]
1000a4ee:    4b1a        ldr      r3, [pc, #104] @ (1000a558 <implant_tick+0xa0>)
1000a4f0:    781b        ldrb     r3, [r3, #0]
1000a4f2:    b303        cbz      r3, 1000a536 <implant_tick+0x7e>
1000a4f4:    4b19        ldr      r3, [pc, #100] @ (1000a55c <implant_tick+0xa4>)
1000a4f6:    781b        ldrb     r3, [r3, #0]
1000a4f8:    2b58        cmp      r3, #88 @ 0x58
1000a4fa:    d01c        beq.n    1000a536 <implant_tick+0x7e>
1000a4fc:    b510        push     {r4, lr}
1000a4fe:    b0c0        sub      sp, #256 @ 0x100
1000a500:    f3ef 8410    mrs      r4, PRIMASK
1000a504:    b672        cpsid    i
1000a506:    22ff        movs     r2, #255 @ 0xff
1000a508:    f10d 0001    add.w    r0, sp, #1
1000a50c:    4611        mov      r1, r2
1000a50e:    f000 fa25    bl       1000a95c <memset>
1000a512:    2358        movs     r3, #88 @ 0x58
1000a514:    f44f 5180    mov.w    r1, #4096 @ 0x1000
1000a518:    4811        ldr      r0, [pc, #68] @ (1000a560 <implant_tick+0xa8>)
1000a51a:    f88d 3000    strb.w   r3, [sp]
1000a51e:    f000 fb6b    bl       1000abf8 <__flash_range_erase_veneer>
1000a522:    f44f 7280    mov.w    r2, #256 @ 0x100
1000a526:    4669        mov      r1, sp
1000a528:    480d        ldr      r0, [pc, #52] @ (1000a560 <implant_tick+0xa8>)
1000a52a:    f000 fb49    bl       1000abc0 <__flash_range_program_veneer>
1000a52e:    f384 8810    msr      PRIMASK, r4
1000a532:    b040        add      sp, #256 @ 0x100
1000a534:    bd10        pop      {r4, pc}
1000a536:    4770        bx       lr
1000a538:    2300        movs     r3, #0
1000a53a:    4904        ldr      r1, [pc, #16] @ (1000a54c <implant_tick+0x94>)
1000a53c:    4a04        ldr      r2, [pc, #16] @ (1000a550 <implant_tick+0x98>)
1000a53e:    700b        strb     r3, [r1, #0]
1000a540:    6013        str      r3, [r2, #0]
1000a542:    4770        bx       lr
1000a544:    600a        str      r2, [r1, #0]
1000a546:    e7d2        b.n      1000a4ee <implant_tick+0x36>
1000a548:    20013718    .word    0x20013718
1000a54c:    20013cf7    .word    0x20013cf7
1000a550:    20013714    .word    0x20013714
1000a554:    20013710    .word    0x20013710
1000a558:    20013cf9    .word    0x20013cf9
1000a55c:    103ff000    .word    0x103ff000
1000a560:    003ff000    .word    0x003ff000

```

The shift `lsls r3, r3, #30` keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) in the carry and sign positions. In `implant_init` the `beq.n` continues to the arm and re-install path only when neither debug bit is set, and falls through to the `0x1000A412` return while a probe is attached. In `implant_tick` the branch is reversed: `bne.n` jumps to the `0x1000A518` clear path while a probe is attached, which clears the armed latch and the beacon count. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```
arm-none-eabi-gdb ACT-X.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A466
(gdb) continue
(gdb) x/4xb 0x103FF000
```

To observe the boot write on the compromised image, break after the flash program at `0x1000A466` (`msr PRIMASK, r4`) in `implant_init`, then read the reserved sector at `0x103FF000` and confirm the first byte is `58`. To observe the tick re-assertion, clear the debug bits (or patch the `ldr.w` at `0x1000A4A4` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `bne`, the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes `0x58` to `0x103FF000`. After the patch, `beq` returns while the gate is clear, so the flash erase and program at `0x1000A456` and `0x1000A462` are never reached and the sector stays blank. The marker is the durable state that re-arms the beacon and masks the hazard on every later boot, and the reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the sabotage marker gate at <code>0x1000A451</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so no sabotage marker is programmed to <code>0x103FF000</code>	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the sabotage marker byte <code>0x58</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA451`; the correct halfword is `d0ef` for `beq.n` and the compromised halfword is `d1ef`, so the on-disk bytes are `EF D0` for the fix and `EF D1` for the compromise.

Only the high byte changes.

- The marker byte is `CHEM_IMPLANT_SABOTAGE_MARKER ( 0x58 )`, the reserved sector is `CHEM_IMPLANT_RESERVE_ADDR ( 0x103FF000 )`, and the marker gate is at `0x20013CF9`. The write uses the real API, `flash_range_erase` and `flash_range_program`, through the veneers at `0x1000ABF8` and `0x1000ABC0`.
- The `DHCSR` address is `CHEM_IMPLANT_DHCSR_ADDR ( 0xE000EDF0 )`; bit 0 is `CHEM_IMPLANT_DHCSR_DEBUGEN ( 0x00000001 )` and bit 1 is `CHEM_IMPLANT_DHCSR_HALT ( 0x00000002 )`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA451` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.
- On a successful disarm with the token the beacon calls `implant_clear_marker`; the documented fix is the patch plus a reserved-sector erase, not the token.

Task 5: Bug #4 The Hazard Command Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x10007620`) the authorization branch is at file offset `0x768D` (VA `0x1000768D`). The corrected image is:

```

10007620 <control_handle_frame>:
10007620:      2107      movs     r1, #7
10007622:      b530      push     {r4, r5, lr}
10007624:      4b1e      ldr      r3, [pc, #120] @ (100076a0 <control_handle_frame+0x80>)
10007626:      b097      sub      sp, #92 @ 0x5c
10007628:      781a      ldrb     r2, [r3, #0]
1000762a:      f88d 1018  strb.w   r1, [sp, #24]
1000762e:      2a00      cmp      r2, #0
10007630:      d033      beq.n    1000769a <control_handle_frame+0x7a>
10007632:      2800      cmp      r0, #0
10007634:      d031      beq.n    1000769a <control_handle_frame+0x7a>
10007636:      2430      movs     r4, #48 @ 0x30
10007638:      a905      add      r1, sp, #20
1000763a:      aa0a      add      r2, sp, #40 @ 0x28
1000763c:      4603      mov      r3, r0
1000763e:      9102      str      r1, [sp, #8]
10007640:      9200      str      r2, [sp, #0]
10007642:      4818      ldr      r0, [pc, #96] @ (100076a4 <control_handle_frame+0x84>)
10007644:      2201      movs     r2, #1
10007646:      a906      add      r1, sp, #24
10007648:      9401      str      r4, [sp, #4]
1000764a:      f000 fa15  bl       10007a78 <envelope_open_hex>
1000764e:      b320      cbz      r0, 1000769a <control_handle_frame+0x7a>
10007650:      9b05      ldr      r3, [sp, #20]
10007652:      2b16      cmp      r3, #22
10007654:      d921      bls.n    1000769a <control_handle_frame+0x7a>
10007656:      f89d 402c  ldrb.w   r4, [sp, #44] @ 0x2c
1000765a:      f8bd 302d  ldrh.w   r3, [sp, #45] @ 0x2d
1000765e:      1e62      subs     r2, r4, #1
10007660:      2a02      cmp      r2, #2
10007662:      b21d      sxth     r5, r3
10007664:      d819      bhi.n    1000769a <control_handle_frame+0x7a>
10007666:      2b10      cmp      r3, #16
10007668:      d817      bhi.n    1000769a <control_handle_frame+0x7a>
1000766a:      f8dd 002f  ldr.w    r0, [sp, #47] @ 0x2f
1000766e:      f8dd 1033  ldr.w    r1, [sp, #51] @ 0x33
10007672:      f8dd 2037  ldr.w    r2, [sp, #55] @ 0x37
10007676:      f8dd 303b  ldr.w    r3, [sp, #59] @ 0x3b
1000767a:      f10d 0c18  add.w    ip, sp, #24
1000767e:      e8ac 000f  stmia.w  ip!, {r0, r1, r2, r3}
10007682:      990a      ldr      r1, [sp, #40] @ 0x28
10007684:      4808      ldr      r0, [pc, #32] @ (100076a8 <control_handle_frame+0x88>)
10007686:      aa06      add      r2, sp, #24
10007688:      f000 f8a2  bl       100077d0 <chem_auth_apply>
1000768c:      b128      cbz      r0, 1000769a <control_handle_frame+0x7a>
1000768e:      4a07      ldr      r2, [pc, #28] @ (100076ac <control_handle_frame+0x8c>)
10007690:      4b07      ldr      r3, [pc, #28] @ (100076b0 <control_handle_frame+0x90>)
10007692:      7014      strb     r4, [r2, #0]
10007694:      801d      strh     r5, [r3, #0]
10007696:      b017      add      sp, #92 @ 0x5c
10007698:      bd30      pop      {r4, r5, pc}
1000769a:      2000      movs     r0, #0
1000769c:      b017      add      sp, #92 @ 0x5c
1000769e:      bd30      pop      {r4, r5, pc}
100076a0:      20013cf5 .word    0x20013cf5
100076a4:      200136e8 .word    0x200136e8
100076a8:      200136cc .word    0x200136cc
100076ac:      20013cf4 .word    0x20013cf4
100076b0:      20013ce6 .word    0x20013ce6

```

Instruction decode. The control ready gate at `0x20013CF5` is loaded at `0x10007604` and a null frame is rejected at `0x10007612`. The sealed frame is opened under the field key at `0x1000762A` by `envelope_open_hex`, and a malformed or too-short body is rejected at `0x1000762E` and `0x10007634`. The command byte is checked against the guarded hazard set by `subs r2, r4, #1` and `cmp r2, #2` at `0x1000763E` and `0x10007640`, and the zone is checked against the 0 to 16 band by `cmp r3, #16` at `0x10007646`. `chem_auth_apply` at `0x10007668` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in `r0`. The

branch at `0x1000768C` decides whether the command may reach the applied command and zone. The correct code rejects a failed or replayed authorization, so the branch at `0x1000768C` must be `cbz (0xB1)` to the `0x1000767A` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013CF4` and the zone at `0x20013CE6`. The condition byte is the high byte at `0x1000768D`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000768D</code>	<code>0x768D</code>	<code>0xB9</code>	<code>cbnz r0, 0x1000767A</code>	<code>0xB1</code>	<code>cbz r0, 0x1000767A</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x768D</code>	<code>0x1000768D</code>	28 B9	28 B1

Why the command now requires authorization. Under the compromised `cbnz`, the verdict is inverted: a failed or replayed authorization falls through to the stores at `0x1000766E`, while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x1000767A`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the command byte and zone are applied. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `CHEM_COMMAND_HAZARD (0x01)`, `CHEM_COMMAND_CLEAR (0x02)`, and `CHEM_COMMAND_ACK (0x03)`, and the zone is checked against the band `0` to `16`. This is the defect that is a policy seam rather than implant behavior, and it is the one a defender would fix first in production.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the hazard command authorization branch at <code>0x1000768D</code>	5	Address and function (<code>control_handle_frame</code>) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why an unauthenticated or replayed hazard command must be rejected	3	The applied command must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x768D`; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928`, so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `chem_auth_apply` (starts at `0x100077D0`) performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- The command body is `seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]`, the `CONTROL_COMMAND_LEN (23)` byte body. The guarded set is `CHEM_COMMAND_HAZARD (0x01)`, `CHEM_COMMAND_CLEAR (0x02)`, and `CHEM_COMMAND_ACK (0x03)`; the zone band is `CHEM_ZONE_MIN (0)` to `CHEM_ZONE_MAX (16)`.

- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the hazard command path is correct. Only the verdict seam was broken.
- The applied command is at `0x20013CF4`, and the applied zone is at `0x20013CE6`. The control ready gate is at `0x20013CF5`, the auth record is at `0x200136CC`, and the field key is at `0x200136E8`.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, `File -> Export Program...`, choose `Binary Format`, and save as `ACT-X_fixed.bin`. The shipped image is 51,172 bytes.

Convert.

```
python uf2conv.py ACT-X_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-X_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-X is 102,912 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-X_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- the terminal no longer forces `SAFE` and the green CLEAR lamp does not mask a declared hazard;
- the beacon gate is clear so the hazard is never masked;
- an unauthenticated command and a replayed captured command are rejected before the command and zone are applied;
- a legitimate authorized command still applies, and the maintenance remote, the acknowledge button, and the fail-safe policy still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Coordinated Beacon	<code>0xA3FD</code>	<code>0x1000A3FD</code>	<code>B9</code>	<code>B1</code>
2	The Persistence	<code>0xA441</code>	<code>0x1000A441</code>	<code>D1</code>	<code>D0</code>
3	The Sabotage Marker	<code>0xA451</code>	<code>0x1000A451</code>	<code>D1</code>	<code>D0</code>
4	The Hazard Command Authorization	<code>0x768D</code>	<code>0x1000768D</code>	<code>B9</code>	<code>B1</code>

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Coordinated Beacon	A terminal publishes a fake all clear while a hazard is live, so a warning system becomes a source of false assurance. Withholding the warning needs no cipher break.
The Persistence	A payload writes durable state that re-installs itself on every boot, so remediation of the running image does not remove it.
The Sabotage Marker	A reserved-sector marker survives a reflash and keeps the device compromised, so incident response must clear durable state, not only the firmware.
The Hazard Command Authorization	An inverted verdict lets an unauthenticated or replayed command change a safety decision, so authorization is defeated without breaking authentication.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-X_fixed.bin from Ghidra	2	Valid patched binary
[PATCH] Converted to ACT-X_fixed.uf2 with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-X.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 51,172 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.
- The complete fix is also a policy: fail safe to the `HAZARD` posture on a lost link, treat a local acknowledge as a request, remove the `SANDBOX_ONLY` implant code path, and clear the reserved sector. The four byte patches close the shipped seams; the policy closes the class.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 chemical store temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it

Device	Pin on device	Pico 2 GPIO	Notes
1602 LCD	SDA	GP2	I2C1, backpack address 0x27
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Siren and vent servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HAZARD, 220 to 330 ohm to GND
Yellow LED	anode	GP17	WATCH, 220 to 330 ohm to GND
Green LED	anode	GP18	CLEAR, 220 to 330 ohm to GND
Acknowledge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The Coordinated Beacon	20
Task 3	Bug #2 The Persistence	20
Task 4	Bug #3 The Sabotage Marker	20
Task 5	Bug #4 The Hazard Command Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational chemical facility network, a safety instrumented system, a building-management system, a public network, a military system, or a third-party device. The implant is benign and confined to the breadboard: it affects only the mock siren and the mock hazard readout, releases on a documented token, and writes only the reserved sector at `0x103FF000` on the same chip. There is no network, no filesystem, and no host impact.

Common Student Mistakes

- Patching the low byte of the branch at `0xA3FC`, `0xA440`, `0xA450`, or `0x768C` instead of the condition byte at `0xA3FD`, `0xA441`, `0xA451`, or `0x768D`.
- Reading the beacon gate backwards and believing the corrected build still masks the hazard.
- Reading the persist gate backwards and believing the corrected build still re-installs the beacon.
- Searching for standalone `implant_reinstall` or `implant_infect` symbols and missing that they are inlined into `implant_init`.
- Confusing the persist gate at `0xA441` with the marker gate at `0xA451`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the sabotage is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the beacon and authorization gates, or `beq` and `bne` on the persistence and marker gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting that the fix is also a policy: fail safe on a lost link and never let a local request silently bypass authorization.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award partial credit for identifying the withheld-warning nature of the beacon without connecting it to the fail-safe policy, or the policy without the beacon.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-X.bin (compromised)
+++ ACT-X_fixed.bin (corrected)

Offset 0x0000768D: B9 -> B1 (cbnz r0, 0x1000769A -> cbz r0, 0x1000769A)
Offset 0x0000A3FD: B9 -> B1 (cbnz r3, 0x1000A402 -> cbz r3, 0x1000A402)
Offset 0x0000A441: D1 -> D0 (bne.n 0x1000A432 -> beq.n 0x1000A432)
Offset 0x0000A451: D1 -> D0 (bne.n 0x1000A432 -> beq.n 0x1000A432)
```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Coordinated Beacon	0xA3FD	0x1000A3FD	0B B9	0B B1
2	The Persistence	0xA441	0x1000A441	F7 D1	F7 D0
3	The Sabotage Marker	0xA451	0x1000A451	EF D1	EF D0
4	The Hazard Command Authorization	0x768D	0x1000768D	28 B9	28 B1

Four defects, four changed bytes in four instructions: the coordinated beacon gate, the persistence re-install gate, the sabotage marker gate, and the authorization verdict. No other byte in either image differs. The CoreDebug DHCSR anti-debug is present and identical in both images, so it is an analysis obstacle and not a fifth patch.